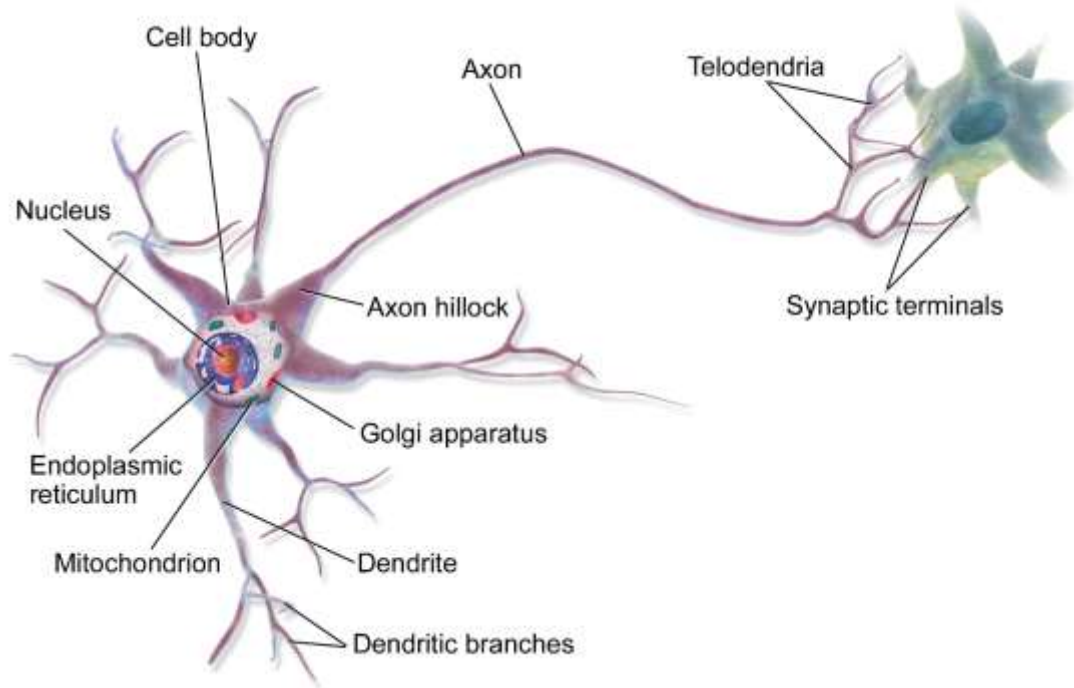


Neural Networks and Deep Learning

Adapted from slides by Dan Jurafsky,
Stanford University

Units in Neural Networks

This is in your brain



Neural Network Unit

This is not in your brain

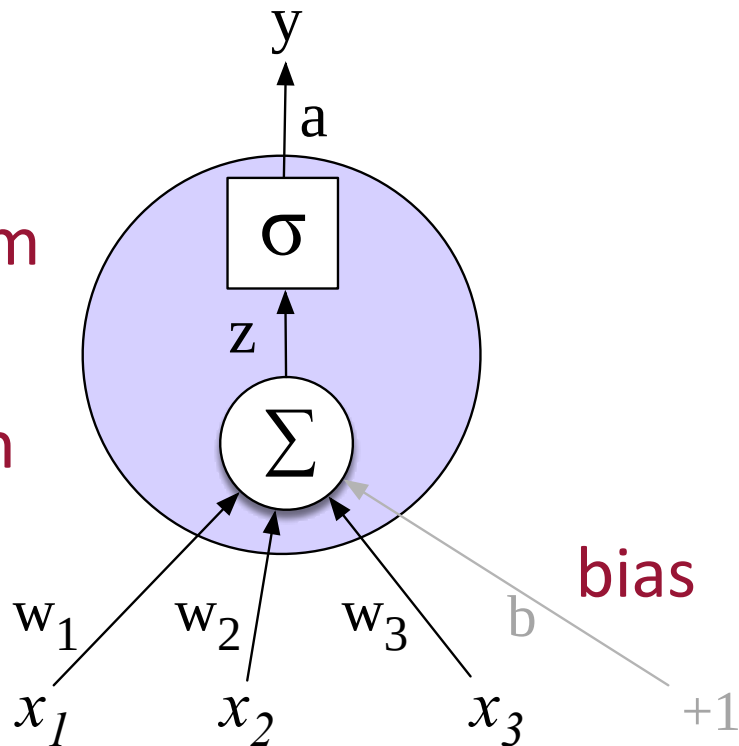
Output value

Non-linear transform

Weighted sum

Weights

Input layer



Neural unit

Take weighted sum of inputs, plus a bias

$$z = w \cdot x + b$$

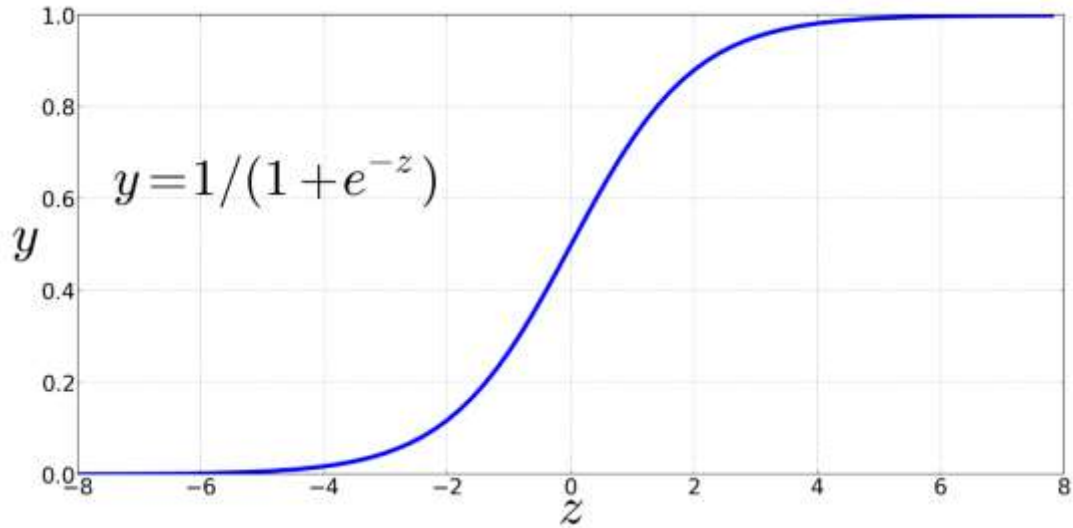
Instead of just using z , we'll apply a nonlinear activation function f :

$$y = a = f(z)$$

Non-Linear Activation Functions

Sigmoid

$$y = s(z) = \frac{1}{1 + e^{-z}}$$



Final function the unit is computing

$$y = s(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Final unit again

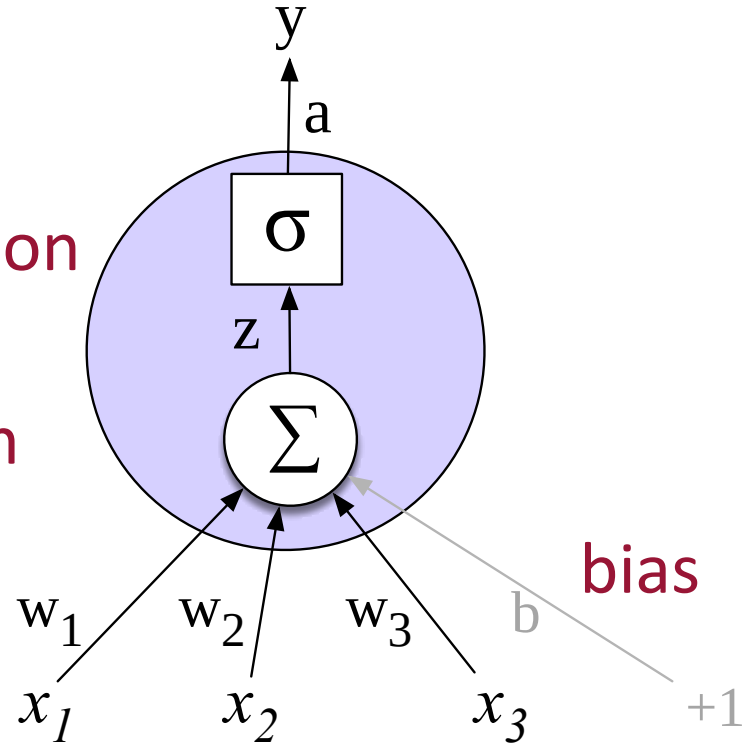
Output value

Non-linear activation function

Weighted sum

Weights

Input layer



An example

Suppose a unit has:

$$w = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What happens with input x :

$$x = [0.5, 0.6, 0.1]$$

$$y = s(w \cdot x + b) =$$

An example

Suppose a unit has:

$$w = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What happens with the following input x ?

$$x = [0.5, 0.6, 0.1]$$

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

An example

Suppose a unit has:

$$w = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What happens with input x :

$$x = [0.5, 0.6, 0.1]$$

1

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

1

$$\frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} =$$

An example

Suppose a unit has:

$$w = [0.2, 0.3, 0.9]$$

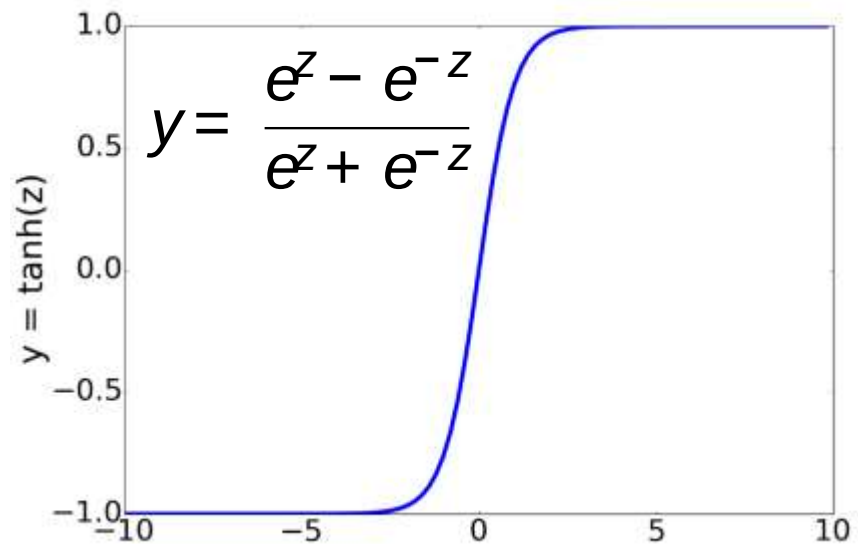
$$b = 0.5$$

What happens with input x :

$$x = [0.5, 0.6, 0.1]$$

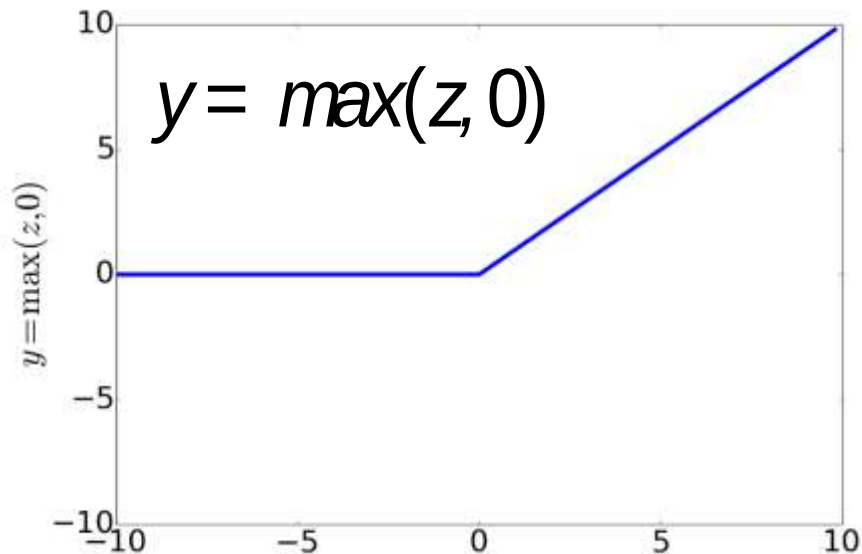
$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Non-Linear Activation Functions besides sigmoid



tanh

Most Common:



ReLU

Rectified Linear Unit

Simple Neural
Networks and
Neural
Language
Models

The XOR problem

Perceptrons

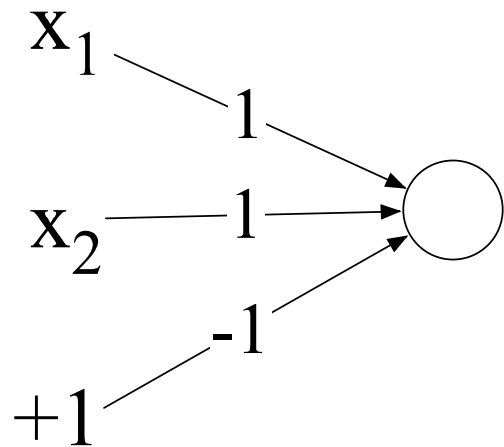
A very simple neural unit

- Binary output (0 or 1)
- No non-linear activation function

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

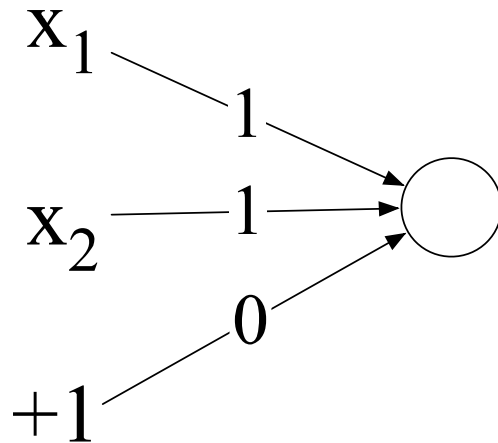
Easy to build AND or OR with perceptrons

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



OR

OR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

Not possible to capture XOR with perceptrons

Why? Perceptrons are linear classifiers

Perceptron equation given x_1 and x_2 , is the equation of a line

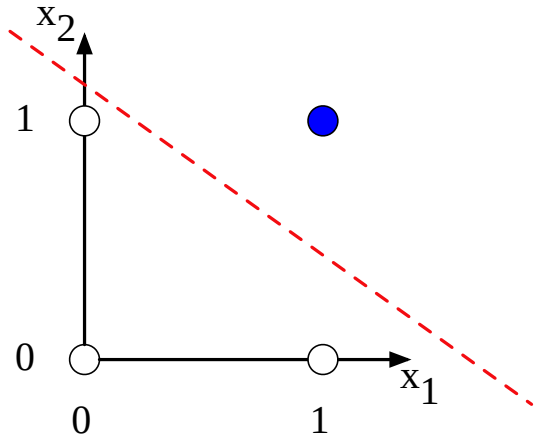
$$w_1x_1 + w_2x_2 + b = 0$$

(in standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$)

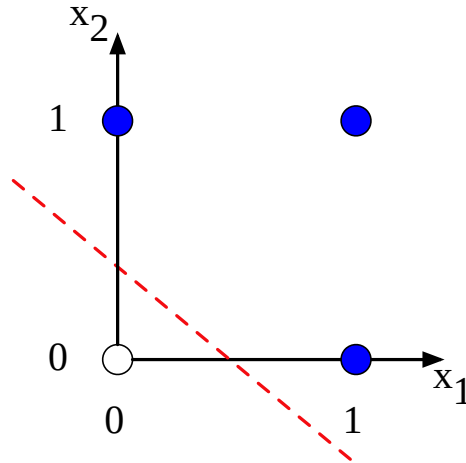
This line acts as a **decision boundary**

- 0 if input is on one side of the line
- 1 if on the other side of the line

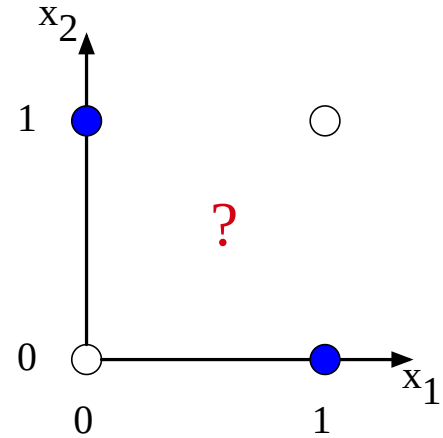
Decision boundaries



a) x_1 AND x_2



b) x_1 OR x_2



c) x_1 XOR x_2

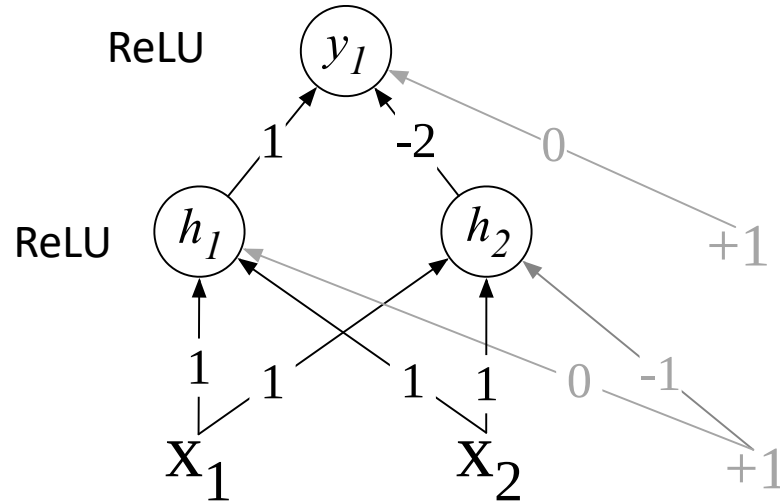
XOR is not a **linearly separable** function!

Solution to the XOR problem

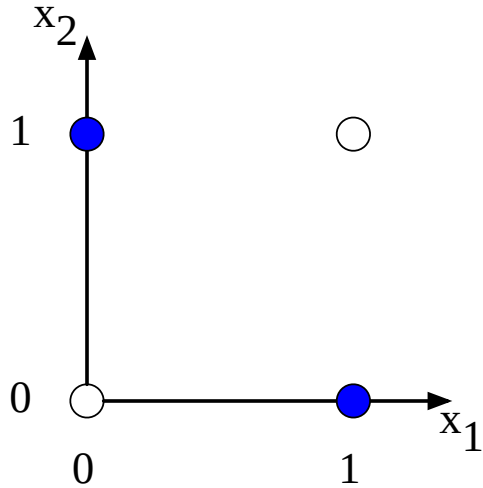
XOR can't be calculated by a single perceptron

XOR can be calculated by a layered network of units.

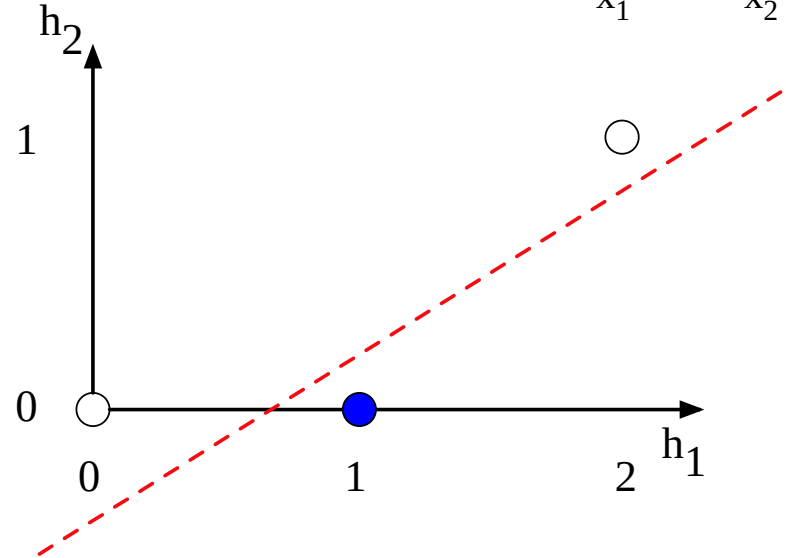
XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



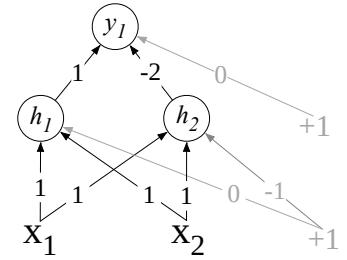
The hidden representation h



a) The original x space



b) The new (linearly separable) h space



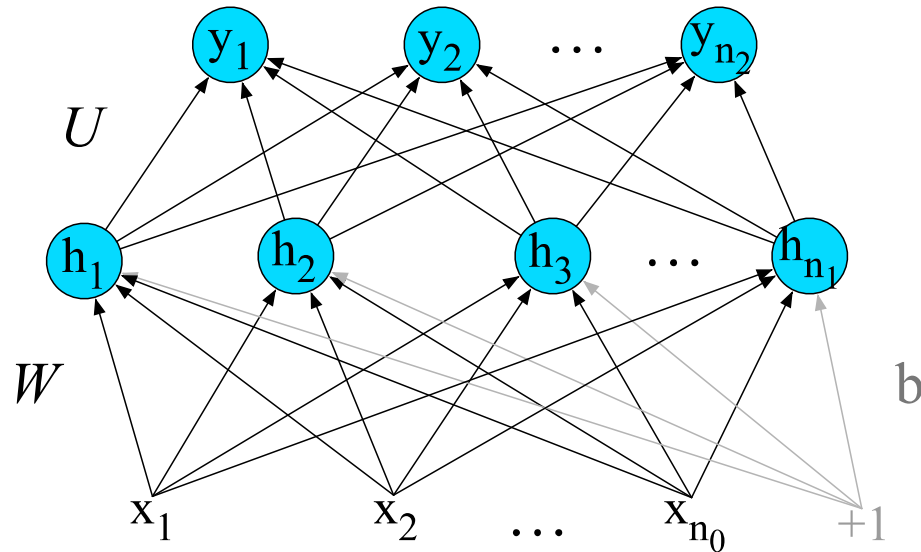
(With learning: hidden layers will learn to form useful representations)

Simple Neural
Networks and
Neural
Language
Models

Feedforward Neural Networks

Feedforward Neural Networks

Can also be called **multi-layer perceptrons** (or **MLPs**) for historical reasons



softmax: a generalization of sigmoid

For a vector z of dimensionality k , the softmax is:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

Example:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

Two-Layer Network with scalar output

Output layer
(σ node)

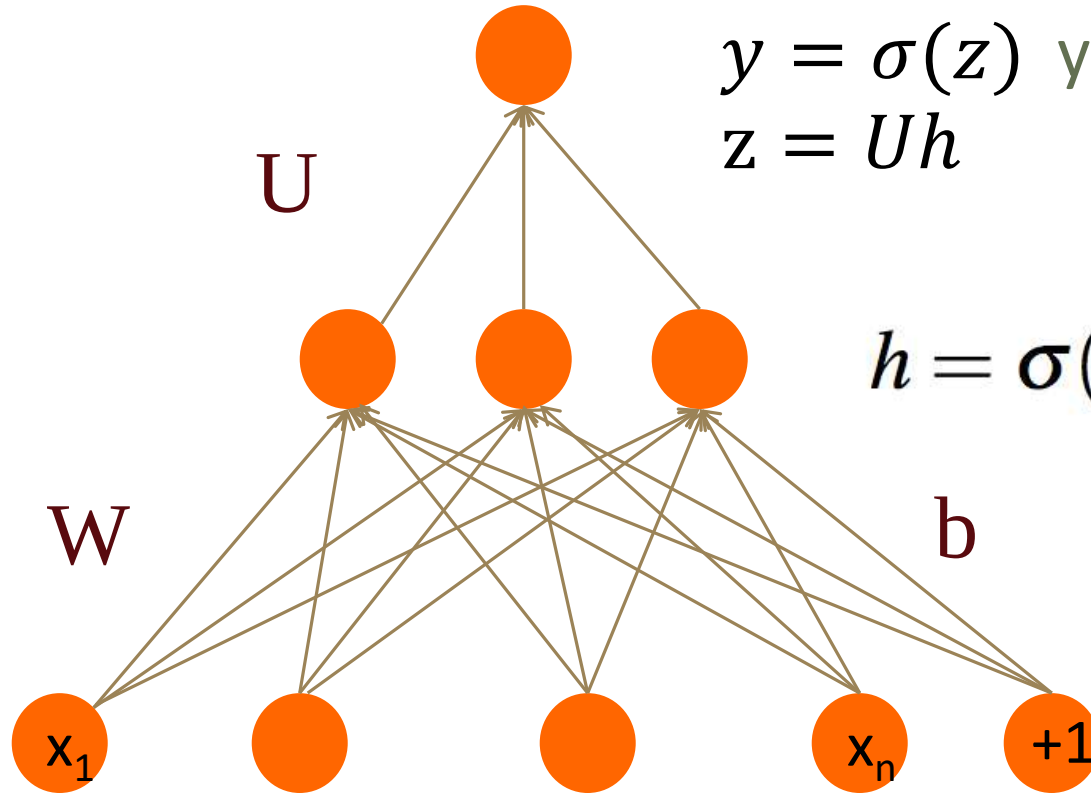
$$y = \sigma(z) \quad y \text{ is a scalar}$$
$$z = Uh$$

hidden units
(σ node)

$$h = \sigma(Wx + b)$$

Could be ReLU
Or tanh

Input layer
(vector)

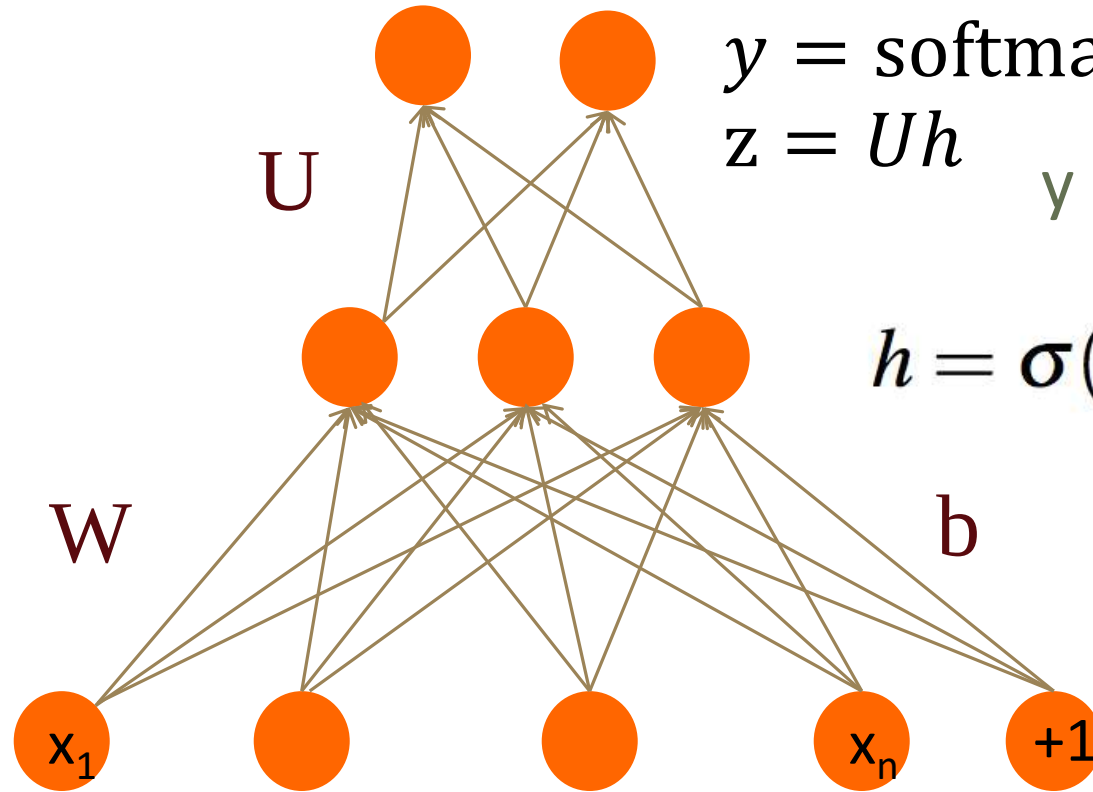


Two-Layer Network with softmax output

Output layer
(σ node)

hidden units
(σ node)

Input layer
(vector)



$$y = \text{softmax}(z)$$

$$z = Uh$$

y is a vector

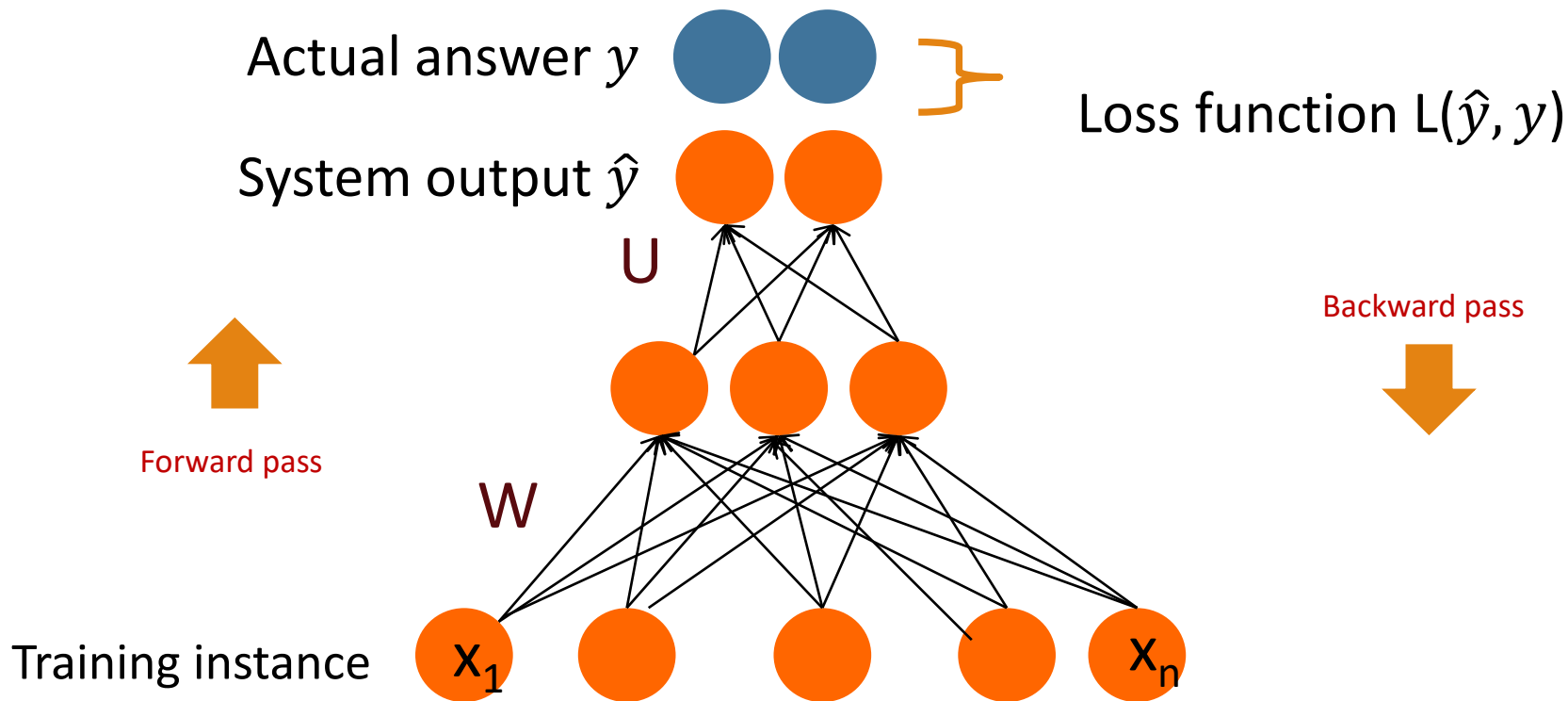
$$h = \sigma(Wx + b)$$

Could be ReLU
Or tanh

Simple Neural
Networks and
Neural
Language
Models

Training Neural Nets: Overview

Intuition: training a 2-layer Network



Intuition: Training a 2-layer network

For every training tuple (x, y)

- Run *forward* computation to find our estimate $\hat{y}$
- Run *backward* computation to update weights:
 - For every output node
 - Compute loss L between true y and the estimated $\hat{y}$
 - For every weight w from hidden layer to the output layer
 - Update the weight
 - For every hidden node
 - Assess how much blame it deserves for the current answer
 - For every weight w from input layer to the hidden layer
 - Update the weight